Orchestrazione SDN di reti QKD multi-dominio: l'interfaccia ETSI GS QKD 018

Diego Carro and Bucci Felice Elia

Corso di Cybersecurity, Politecnico di Torino
Quantum Engineering

Novembre 2026

Abstract

La distribuzione quantistica di chiavi (QKD) permette a due parti di concordare una chiave segreta con una sicurezza fondata sulle leggi della fisica anziché sulla difficoltà computazionale. Portarla dal laboratorio alla rete di un operatore richiede però di gestirla come qualunque altra risorsa di rete: scoprirne la topologia, monitorarne lo stato, allocarne la capacità ai servizi e coordinarla con le reti classiche. Il gruppo ETSI ISG QKD ha affrontato il problema per gradi. La GS QKD 002 individua i casi d'uso e i requisiti di integrazione; la GS QKD 015 definisce il modello con cui un controller SDN vede e configura i nodi QKD di un dominio; la GS QKD 018 definisce l'interfaccia tra un *SDN orchestrator* e il controller di un dominio QKD, abilitando il provisioning end-to-end tra più domini e tra rete QKD e rete di trasporto ottico. Questo lavoro ripercorre la catena in modo ragionato: descrive le componenti di una rete SD-QKD e la struttura di un modello YANG, analizza in dettaglio la GS QKD 018 (casi d'uso, modello informativo, moduli di topologia, inventario e connettività, workflow multi-dominio, sicurezza), ne mostra l'analogia con il framework ACTN dell'IETF (RFC 8453) e la confronta con tre esperienze reali: il testbed multi-vendor KOREN di SK Telecom, un prototipo gerarchico basato su ONOS sviluppato al Politecnico di Milano e la rete MadQCI di Madrid. Si conclude con i limiti attuali dello standard e le direzioni di ricerca aperte.

Parole chiave: QKD, SDN, orchestrazione, ETSI GS QKD 015, ETSI GS QKD 018, YANG, RESTCONF, multi-dominio, gestione delle chiavi.

Indice

1 Stato dell'arte **2**

2 Introduzione **3**

2.1 ETSI GS QKD 002: i casi d'uso 5

2.1.1 La QKD come primitiva crittografica 5

2.1.2 Dove si inserisce la QKD nello stack di rete 5

2.1.3 I sei casi d'uso 6

2.1.4 Che cosa lascia in eredità 7

2.2 ETSI GS QKD 015: il controller SDN e la rete SD-QKD 8

2.2.1 Perché un controllo software-defined 8

2.2.2 Le componenti di una rete SD-QKD 8

2.2.3 Anatomia di un modello YANG 10

2.2.4 Il modello del nodo SD-QKD 12
2.2.5 Notifiche 14
2.2.6 I workflow 14
2.2.7 Protocolli e sicurezza 15
2.2.8 Dal dominio singolo all'orchestrazione 16

3 ETSI GS QKD 018 **16**
3.1 Use case della QKD 018 16
3.2 Il modello informativo 17
3.3 I moduli YANG 18
3.4 Workflow di provisioning multi-dominio 20
3.5 Protocolli e sicurezza 21

4 L'analogia con ACTN (RFC 8453) **21**

5 Casi d'uso reali **22**
5.1 SK Telecom: orchestrazione multi-vendor su KOREN 22
5.2 Politecnico di Milano: controllo gerarchico con ONOS 22
5.3 MadQCI: la rete di Madrid 22

6 Limiti e prospettive **23**

7 Conclusioni **23**

Bibliografia **23**

1 Stato dell'arte

Fondamenti di QKD. La letteratura sulla QKD è ormai consolidata. La review di Pirandola et al. [1] copre i protocolli a variabili discrete (BB84 e le sue varianti con *decoy state*), quelli a variabili continue e le dimostrazioni di sicurezza. Da qui emergono i due vincoli che guidano tutta l'ingegneria di rete: il tasso di chiave decresce esponenzialmente con la distanza e un singolo collegamento in fibra commerciale non supera le poche centinaia di chilometri.

Dal collegamento alla rete. Mehic et al. [2] e Cao et al. [3] descrivono il passaggio dal collegamento punto-punto alla *rete* QKD: architettura a strati, *trusted relay*, gestione e instradamento delle chiavi. Il white paper GÉANT [4] offre una sintesi leggibile dello stesso quadro e degli standard ETSI e ITU-T. Il problema dell'instradamento, che diventa centrale quando si orchestra una rete, è trattato in [5].

Standardizzazione. Due organismi hanno prodotto specifiche complementari. L'ETSI ISG QKD ha iniziato nel 2010 con i casi d'uso [6] e ha poi definito le interfacce verso le applicazioni [7, 8], verso il controller SDN [9], verso l'orchestrator [10] e tra sistemi di gestione delle chiavi [11]. L'ITU-T, con la serie Y.3800, ha definito l'architettura di riferimento delle reti QKD (QKDN) [12], il loro controllo e la loro gestione [13] e il controllo tramite SDN [14].

SDN applicato alla QKD. Il paradigma *Software-Defined Networking* [15] separa il piano di controllo dal piano dati e accentra le decisioni in un controller programmabile. Il gruppo UPM/Telefónica ha mostrato per primo l'integrazione di risorse QKD in una rete ottica controllata via SDN, sia sperimentalmente [16] sia in un tutorial sistematico [17]; da questo filone nasce la GS QKD 015.

Orchestrazione e multi-dominio. L'interfaccia orchestrator–controller è specificata dalla GS QKD 018 [10] e dai relativi moduli YANG [18]. Il primo caso reale documentato è quello di Sim et al. [19] (SK Telecom), che orchestrano due domini di vendor diversi sul testbed KOREN. La tesi di Vicario [20] realizza un prototipo gerarchico completo con ONOS e RESTCONF. La rete MadQCI [21] usa estesamente la GS QKD 015 ma indica l'orchestrazione GS QKD 018 come attività ancora in corso. Lavori più recenti propongono applicazioni SDN per la riconfigurazione dinamica dei link [22], criteri di selezione dei cammini e instradamento inter-dominio che preserva la riservatezza della topologia [23] e modelli ibridi di consegna delle chiavi tra reti regionali [24].

Contributo. Rispetto a queste fonti, il presente lavoro offre una lettura unitaria e ragionata della catena GS QKD 002 $\rightarrow$ 015 $\rightarrow$ 018: mostra come i requisiti dei casi d'uso si traducano nelle componenti di una rete SD-QKD, spiega la struttura dei modelli YANG che le descrivono e analizza come la GS QKD 018 li astragga per l'orchestrazione multi-dominio.

2 Introduzione

Questa sezione costruisce il vocabolario necessario per leggere le specifiche ETSI: parte dalla QKD come primitiva crittografica, passa alla rete QKD con i suoi nodi fidati e la sua architettura a strati, richiama i concetti essenziali di SDN e arriva infine ai due documenti che precedono la GS QKD 018: la GS QKD 002 (§2.1) e la GS QKD 015 (§2.2).

La distribuzione quantistica di chiavi.

Definizione 2.1 (QKD)**.** La *Quantum Key Distribution* è una procedura che, tramite il trasporto di stati quantistici, consente a due parti remote di concordare stringhe di bit segrete condivise, con una sicurezza basata sull'entanglement o sull'impossibilità di clonare o misurare perfettamente stati quantistici sconosciuti [10].

L'idea si capisce bene con BB84. Alice codifica bit casuali su singoli fotoni in una di due basi scelte a caso; Bob misura in una base anch'essa casuale; su un canale classico *autenticato* i due confrontano le basi e tengono solo i bit misurati nella base giusta (*sifting*). Un intercettatore che misura i fotoni introduce inevitabilmente errori, quantificati dal *Quantum Bit Error Rate*

$$\mathrm{QBER} = \frac{N_{\mathrm{errori}}}{N_{\mathrm{sifted}}}. \tag{1}$$

Se il QBER resta sotto una soglia (circa l'11% per BB84 contro attacchi collettivi), correzione d'errore e *privacy amplification* distillano una chiave segreta; altrimenti il blocco viene scartato. La grandezza che interessa alla rete è il *secret key rate* (SKR), cioè i bit di chiave sicura prodotti al secondo. Poiché la trasmittanza della fibra decade come $\eta = 10^{-\alpha L/10}$, con $\alpha \approx 0{,}2$ dB/km, anche l'SKR cala rapidamente con la lunghezza L del collegamento.

Due osservazioni, che ritroveremo nei casi d'uso della GS QKD 002, sono fondamentali. La QKD *distribuisce* chiavi ma non cifra né autentica: va sempre combinata con un cifrario (one-time pad o AES) e con un meccanismo di autenticazione. Inoltre il canale classico deve essere autenticato fin dal primo scambio, il che richiede un piccolo segreto pre-condiviso: la QKD è quindi, a rigore, un meccanismo di *key growing*.

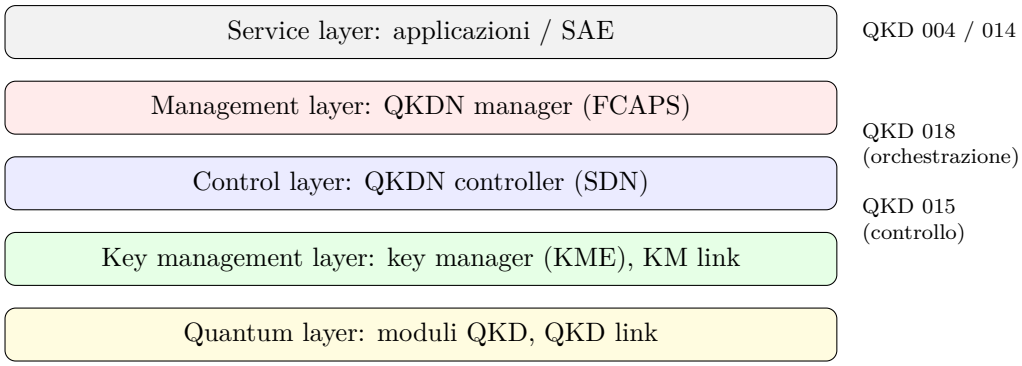


Figura 1: Architettura a strati di una rete QKD secondo ITU-T Y.3800 [12], con a destra la collocazione indicativa delle specifiche ETSI discusse nel testo.

Dal link alla rete: il trusted node. Un link QKD copre al massimo qualche centinaio di chilometri e non esistono ancora ripetitori quantistici pratici. Le reti reali collegano quindi i link in serie tramite nodi fidati.

Definizione 2.2 (Trusted node e key relay)**.** Un *trusted node* è un nodo, collocato in un perimetro fisicamente protetto, che condivide chiavi QKD con entrambi i vicini. Per trasportare una chiave K da A a C passando per B, il nodo A invia $c = K \oplus K_{AB}$; B calcola $K = c \oplus K_{AB}$ e inoltra $c' = K \oplus K_{BC}$. Ogni hop è cifrato con *one-time pad*, ma K compare in chiaro dentro ciascun nodo intermedio, che deve quindi essere fidato.

Si noti che il relay consuma chiave: per trasportare $|K|$ bit lungo un cammino p ogni link $\ell \in p$ spende $|K|$ bit della propria chiave locale. Questo semplice fatto rende la chiave una *risorsa di rete* da pianificare, ed è il motivo per cui le specifiche successive parlano di tassi disponibili, consumi e cammini.

L'architettura a strati ITU-T. La raccomandazione ITU-T Y.3800 [12] organizza una rete QKD (QKDN) in strati sovrapposti (Fig. 1): il *quantum layer* con i moduli QKD; il *key management layer* con i key manager che memorizzano e inoltrano le chiavi; il *control layer*, che prende decisioni operative (instradamento, relay, configurazione dei link); il *management layer*, che supervisiona il tutto; e il *service layer*, dove le applicazioni consumano le chiavi. In questo schema si distinguono due tipi di collegamento.

Definizione 2.3 (QKD link e KM link)**.** Un *QKD link* connette due moduli QKD ed è composto da canale quantistico e canale classico: è il luogo in cui la chiave viene *generata*. Un *KM link* connette due *key manager* ed è il canale classico, protetto con chiavi QKD, lungo il quale la chiave viene *inoltrata* (key relay).

Allo stesso modo si distinguono due ruoli spesso confusi. Il *QKDN controller* agisce sulla rete in tempo reale: decide i cammini di relay, attiva o disattiva link, reagisce ai guasti. Il *QKDN manager* ha un compito di supervisione: raccoglie informazioni FCAPS (*Fault, Configuration, Accounting, Performance, Security*) e applica politiche. Le specifiche ETSI che vedremo si collocano nel piano di controllo: la GS QKD 015 tra controller e nodi, la GS QKD 018 tra controller e orchestrator.

Le applicazioni non vedono la parte quantistica. Un *Secure Application Entity* (SAE) chiede chiavi al *Key Management Entity* (KME) del proprio nodo tramite le API GS QKD 004 [7] o GS QKD 014 [8]; due SAE che vogliono comunicare ricevono la stessa chiave dai rispettivi KME, identificata da un *key ID*.

Software-Defined Networking. In una rete tradizionale ogni apparato decide da solo come instradare il traffico. SDN [15] separa invece tre piani: il *piano dati*, fatto di dispositivi che eseguono; il *piano di controllo*, accentrato in un controller con la vista globale della rete; il *piano di gestione e applicazione*, che esprime gli obiettivi. Il controller parla con i dispositivi tramite un'interfaccia *southbound* (SBI) e offre i propri servizi verso l'alto tramite un'interfaccia *northbound* (NBI). Quando i domini sono più di uno, si aggiunge sopra i controller un *orchestrator*, che coordina servizi end-to-end senza conoscere i dettagli di ciascun dominio.

Una rete QKD ha tutte le caratteristiche che rendono utile SDN: risorse scarse e variabili (la chiave), vincoli fisici stringenti (attenuazione, coesistenza con canali classici), apparati di vendor diversi e domande che cambiano nel tempo. Le due specifiche che seguono descrivono perché e come l'ETSI ha applicato questo paradigma alla QKD.

2.1 ETSI GS QKD 002: i casi d'uso

La GS QKD 002 V1.1.1 (giugno 2010) [6] è il primo documento pubblicato dall'ISG QKD. Il suo obiettivo non è tecnico in senso stretto: prima di standardizzare *come* realizzare la QKD, il gruppo voleva chiarire *dove* serve, *chi* la usa e *quali requisiti* deve rispettare per entrare nelle infrastrutture esistenti. Il documento ragiona in tre passi: colloca la QKD tra le primitive crittografiche, individua i livelli dello stack di rete in cui può essere inserita e descrive sei casi d'uso con i relativi requisiti. Seguire questo ragionamento è utile perché mostra da dove nascono le esigenze che, dodici anni dopo, porteranno alle GS QKD 015 e 018.

2.1.1 La QKD come primitiva crittografica

Il documento parte da un sistema QKD tipico: due unità, di solito montate in rack da 19", collegate da un canale quantistico in fibra o in spazio libero lungo fino a circa 100 km, che producono alle due estremità la stessa sequenza di bit casuali. La clausola 4 chiarisce subito che questo risolve *un solo* problema crittografico.

Un sistema sicuro ha bisogno di tre primitive: *cifratura*, *distribuzione delle chiavi* e *autenticazione dei messaggi*. La QKD copre solo la seconda; per la cifratura servono one-time pad o cifrari simmetrici come AES, per l'autenticazione servono MAC (per esempio basati su hashing universale di tipo Wegman–Carter) o firme digitali. Ne segue che la QKD è un *complemento* alla crittografia classica, non un suo sostituto, e che la sua sicurezza complessiva dipende anche dalle altre componenti del sistema. Il documento sottolinea inoltre che il canale pubblico della QKD va autenticato con un segreto pre-distribuito.

2.1.2 Dove si inserisce la QKD nello stack di rete

La clausola 6 descrive in quale livello dello stack una chiave QKD può essere consumata. La scelta del livello determina chi deve essere modificato e quanto l'integrazione è trasparente.

Livello di collegamento (L2)
La chiave QKD alimenta lo scambio di chiavi di PPP, le chiavi di IEEE 802.1AE (MACsec) oppure un *QKD link encryptor*, cioè un cifratore di linea che consuma direttamente le chiavi prodotte dal link e crea un tunnel cifrato tra due siti. È l'opzione più trasparente per le applicazioni.

Livello di rete (L3)
Una versione modificata di IKE usa la chiave QKD come segreto condiviso per IPsec.

Livello di trasporto (L4)
La chiave QKD sostituisce le chiavi di sessione di TLS/SSL o viene usata al posto degli HMAC per l'autenticazione.

Livello applicativo
Le applicazioni usano la chiave per autenticare gli utenti o cifrare il contenuto scambiato tra le proprie istanze.

Il punto concettuale è che, qualunque sia il livello, serve un componente che *consuma* chiavi e uno che le *fornisce*: è la separazione che più tardi diventerà SAE e KME, con un'interfaccia standard (GS QKD 004 e 014) tra i due.

2.1.3 I sei casi d'uso

La clausola 7 descrive sei casi d'uso con una struttura fissa: obiettivo, descrizione, attori, benefici, considerazioni operative e di qualità del servizio. Li riassumiamo seguendo un filo logico: si passa dal collegamento singolo alla rete, e dalla rete privata alla rete di un operatore.

UC1 – Offsite backup / business continuity. Un'impresa replica i dati da un sito primario a un sito di backup (backup giornalieri, snapshot, mirroring di SAN, replica di database). Un singolo link QKD punto-punto alimenta un cifratore di linea su Ethernet o Fibre Channel; la fibra può essere dedicata (*dark fibre*) o condivisa in WDM o TDM. I requisiti emersi sono: rinnovo della chiave tanto frequente quanto il tasso di generazione consente, gestione delle fasi di ricalibrazione durante trasferimenti continui, un segreto iniziale all'installazione e alta velocità di cifratura. Il documento nota che la sicurezza complessiva non è incondizionata, perché dipende anche dal cifratore.

UC2 – Rete metropolitana aziendale. Più sedi di un'impresa o di un ente pubblico sono collegate ai data center, attraverso i quali passa anche il traffico tra sedi. Ogni collegamento è protetto da un QKD link encryptor che cifra e autentica a livello 2, in modo trasparente per le applicazioni. Emergono due esigenze nuove: link diversi hanno volumi e tempi di traffico diversi, quindi *requisiti di QoS diversi*, e i trasferimenti possono essere pianificati, periodici o attivati da soglie. La variante *QKD Secured Key Server* prevede un server centrale che distribuisce chiavi e certificati ai client dell'area metropolitana su canali cifrati con one-time pad e autenticati con Wegman–Carter: è un primo esempio di servizio di chiavi offerto da una rete.

UC3 – Controllo di infrastrutture critiche. Il caso protegge il traffico SCADA di un'infrastruttura critica, con l'esempio di una rete ferroviaria: qui conta più l'*autenticità* dei comandi (scambi, segnali) che la loro riservatezza. I nodi della WAN privata stanno in stazioni con perimetro controllato, mentre la fibra è fuori dal perimetro. Il documento propone due architetture: link QKD singoli tra nodi, con buffer e inoltro delle chiavi gestiti nei nodi, oppure una *rete QKD con gestione delle chiavi integrata* e un'interfaccia dedicata verso le applicazioni. Si chiedono la massima disponibilità, con precauzioni per ricalibrazioni, guasti e denial of service, e la possibilità di inizializzare nuovi link ridondanti con segreti ottenuti lungo un cammino alternativo. È il primo caso in cui compaiono, insieme, key relay, gestione centralizzata e ridondanza dei cammini.

UC4 – Protezione del backbone. Una fibra del backbone metropolitano di un operatore, tipicamente un anello WDM con OADM o ROADM, trasporta un canale quantistico accanto ai canali dati. Le chiavi proteggono il traffico di controllo e gestione o vengono offerte ai clienti. Gli attori sono due: chi gestisce l'infrastruttura e chi offre servizi sopra di essa; i nodi del backbone

sono fidati. I requisiti sono i più vicini a quelli di un operatore: il sistema QKD non deve ridurre il throughput del sistema WDM di oltre il 10%, deve essere compatibile con fibre, giunti e connettori comuni, la sua gestione deve *integrarsi in quella del sistema WDM* e installazione e manutenzione devono essere possibili a personale senza competenze di fisica quantistica.

UC5 – Rete di accesso ad alta sicurezza. In una rete ottica passiva (PON) per Fiber-to-the-Home, il traffico verso gli utenti è trasmesso in broadcast a tutte le ONU, quindi va cifrato per impedire che un utente legga i dati di un altro. Un OLT serve da 32 a 128 ONU e il canale quantistico segue lo stesso percorso di quello classico. La configurazione è fortemente asimmetrica: un solo rivelatore all'OLT, più costoso, serve molte sorgenti alle ONU. Le distanze brevi permettono tassi elevati, necessari perché la capacità di generazione è condivisa tra tutti gli utenti. La variante *QKD Authenticated Sensor Network* applica lo stesso schema a una stazione centrale con molti sensori e attuatori.

UC6 – Servizi a lunga distanza (long-haul). Due stazioni a terra A e B molto distanti, per esempio sulle due sponde di un oceano, vogliono condividere una chiave senza fidarsi di nodi intermedi a terra. Un nodo mobile C (satellite, aereo o piattaforma ad alta quota) stabilisce un segreto a con A e poi un segreto b con B, e invia a B il valore $a \oplus b$ su canale classico; B ricava a. È un key relay in cui il nodo fidato è C. I satelliti in orbita bassa operano a circa 300–800 km di quota e passano sopra le stazioni circa una volta al giorno; l'orbita geostazionaria non è praticabile. Meteo e atmosfera possono ridurre o annullare il tasso, quindi servono canali ridondanti.

La Tab. 1 riassume i sei casi.

Tabella 1: I casi d'uso della GS QKD 002 e gli elementi che anticipano le specifiche successive.

Caso	**Configurazione**	**Esigenza che anticipa**
UC1 Backup	link punto-punto + cifratore di linea	interfaccia chiave–cifratore
UC2 Metro aziendale	più link, server di chiavi	QoS per applicazione
UC3 Infrastrutture critiche	rete con key relay e gestione integrata	KMS, ridondanza, controllo centralizzato
UC4 Backbone	canale quantistico in WDM	coesistenza con la rete ottica, gestione integrata
UC5 Accesso PON	punto-multipunto	condivisione della capacità di chiave
UC6 Long-haul	relay tramite nodo mobile	trusted node, ridondanza

2.1.4 Che cosa lascia in eredità

La GS QKD 002 è un documento di prima edizione: la clausola sui requisiti generali e le caratteristiche funzionali dei singoli casi sono indicate come non ancora disponibili. Il suo valore sta nell'aver fissato le domande. Letti in sequenza, i casi d'uso mostrano che la QKD utile non è il singolo link ma una *rete*, che richiede: un sistema di gestione delle chiavi che le memorizzi e le inoltri (UC3, UC6); un modo per esprimere requisiti di servizio diversi per applicazioni diverse (UC2); la convivenza con la rete ottica dell'operatore, con una gestione integrata in quella esistente e utilizzabile da personale non specializzato (UC4). Le prime due esigenze troveranno risposta nella GS QKD 015, la terza, nella forma del coordinamento tra rete QKD e rete di trasporto, nella GS QKD 018.

2.2 ETSI GS QKD 015: il controller SDN e la rete SD-QKD

La GS QKD 015 (V1.1.1 nel 2021, V2.1.1 nell'aprile 2022) [9] definisce l'interfaccia di controllo tra un controller SDN e i nodi di una rete QKD, cioè la *southbound interface* del piano di controllo QKD. La seconda edizione, quella considerata qui, è stata allineata alla GS QKD 018: i moduli YANG della 018 importano direttamente quelli della 015. Capire bene la 015 è quindi il prerequisito per capire la 018.

La sezione procede in quattro passi: perché serve un controllo software; quali sono, una per una, le componenti di una rete SD-QKD; come è fatto in generale un modello YANG; come la 015 usa YANG per descrivere il nodo, con i workflow e le considerazioni di sicurezza che ne derivano.

2.2.1 Perché un controllo software-defined

La specifica parte da una constatazione: integrare la QKD in una rete di telecomunicazioni è oggi un'operazione manuale. Ogni nuovo sistema QKD viene installato a mano, di solito su una fibra scura dedicata che collega due siti, formando un link fisso. Questo approccio ha tre limiti. Non scala, perché ogni modifica richiede intervento umano. Spreca risorse, perché la capacità di chiave di un link non può essere spostata dove serve. Infine non dialoga con la rete ottica, che invece è sempre più spesso disaggregata e controllata da software.

L'architettura proposta permette a un controller di orchestrare centralmente le risorse QKD: ottimizzare l'allocazione di chiave per link in base alla domanda, creare automaticamente collegamenti diretti o multi-hop, coordinare canali quantistici e classici anche quando condividono la stessa fibra. In particolare l'astrazione definita dalla specifica mira a:

- gestire centralmente le risorse QKD in base a domande, capacità e stato della rete;
- aggregare i sistemi QKD di uno stesso perimetro sotto un'unica gestione delle chiavi, per riconoscere meglio le domande;
- nascondere la complessità dei parametri di basso livello di ogni tecnologia, delegandoli al nodo;
- ottimizzare la distribuzione dei link in base alla QoS delle applicazioni e alle statistiche di generazione;
- coordinare canali quantistici e classici, condividano o meno lo stesso mezzo fisico.

Sono, in forma tecnica, le esigenze emerse dai casi UC2, UC3 e UC4 della GS QKD 002.

2.2.2 Le componenti di una rete SD-QKD

Conviene costruire la rete dal basso, aggiungendo un'astrazione alla volta. La Fig. 2 mostra il risultato.

1. Modulo, sistema e link QKD. L'unità fisica di base è il *modulo QKD*: l'insieme di hardware e software che esegue una parte di un protocollo QKD. Due moduli collegati formano un *sistema QKD*, e il collegamento tra loro (canale quantistico più canale classico) è il *QKD link*. Fin qui siamo nel quantum layer della Fig. 1: i moduli sono specifici del vendor e della tecnologia (variabili discrete o continue, codifica, lunghezza d'onda), e non sono interoperabili tra loro.

2. Interfaccia QKD. Il controller non ha bisogno di sapere come funziona internamente un modulo, ma solo che cosa offre alla rete. La 015 introduce quindi un'astrazione.

Definizione 2.4 (Interfaccia QKD)**.** Un'*interfaccia QKD* (`qkdi`) è la rappresentazione astratta di un modulo QKD all'interno di un nodo: ne espone solo gli attributi rilevanti per la rete, come tecnologia, modello, intervallo di lunghezze d'onda, attenuazione massima supportata e punto di attacco a uno switch ottico.

L'analogia è quella delle porte di uno switch: un nodo con tre moduli QKD appare al controller come un apparato con tre interfacce. Poiché moduli di vendor diversi non interoperano, il modello impone di dichiarare tecnologia e modello di ciascuna interfaccia: il controller abbinerà solo interfacce compatibili.

3. Nodo SD-QKD. Le interfacce che stanno nello stesso luogo sicuro vengono raggruppate.

Definizione 2.5 (Nodo SD-QKD)**.** Un *nodo SD-QKD* è l'aggregazione di uno o più moduli QKD situati in un luogo sicuro e gestiti da un unico *SDN agent*, che li espone al controller come un singolo elemento di rete con più interfacce.

La specifica fissa una serie di requisiti che danno al nodo la sua forma. Un nodo SD-QKD *deve*: comprendere almeno un modulo QKD, esposto come interfaccia; trovarsi in un luogo sicuro (un luogo può ospitare più nodi); contenere un *sistema di gestione delle chiavi* che aggrega il materiale prodotto da tutte le associazioni di chiave; offrire alle applicazioni un *unico punto di accesso* alle chiavi tramite API; collegarsi ad almeno un controller SDN con protocolli standard, per discovery, controllo e telemetria. Inoltre *dovrebbe* esporre al controller le informazioni su applicazioni, interfacce, link e requisiti del canale classico (per esempio l'attenuazione accettabile).

All'interno del nodo si riconoscono quindi quattro componenti logiche:

- i **moduli QKD**, che generano chiave con i nodi remoti;
- il **key manager** (KMS), che riceve le chiavi dai moduli, le memorizza in uno o più *key store* logici, le inoltra in caso di relay e registra le applicazioni con la loro QoS;
- l'**interfaccia verso le applicazioni**, cioè la key delivery API (GS QKD 004 o 014), raggiungibile a un indirizzo IP e porta noti;
- l'**SDN agent**, che traduce lo stato di tutto ciò in un modello YANG, lo espone al controller, riceve configurazioni e invia notifiche.

4. Key association link. Tra due nodi la grandezza interessante non è il canale quantistico in sé ma il fatto che i due nodi condividano chiave. La 015 modella proprio questo.

Definizione 2.6 (Key association link)**.** Un *key association link* (`qkd_link`) è un'associazione logica di chiave tra due nodi SD-QKD remoti. È di tipo `PHYS` (diretto) se esiste un canale quantistico tra i due nodi, di tipo `VIRT` (virtuale) se la chiave è ottenuta tramite key relay attraverso nodi fidati intermedi.

Un link fisico corrisponde a un QKD link della terminologia ITU-T; un link virtuale corrisponde invece a una catena di KM link sopra più QKD link. Due dettagli rendono il modello coerente. Il primo: un link virtuale *consuma* chiave dai link fisici su cui poggia, e per questo viene registrato come *applicazione interna* con una banda riservata (`virt_bandwidth`). Il secondo: ogni nodo intermedio di un link virtuale conosce il nodo precedente e il successivo (`virt_prev_hop`, `virt_next_hop`), quindi il cammino di relay è configurato hop per hop dal controller. Perché un nodo possa fare da relay deve dichiararlo con la capability `key_relay_mode_enable`.

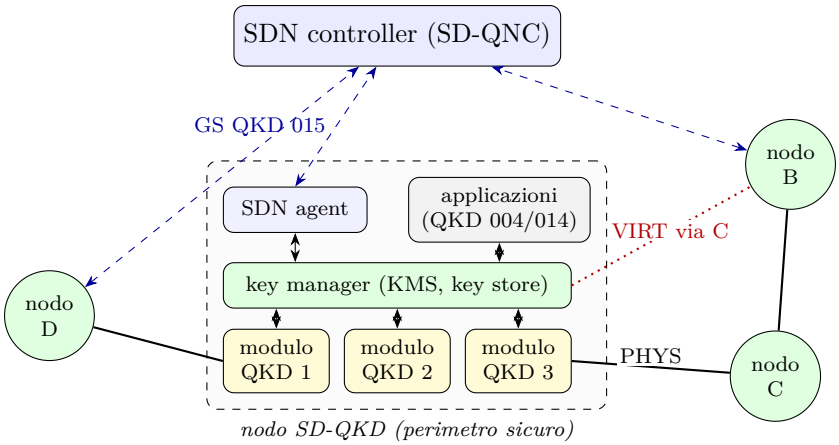


Figura 2: Una rete SD-QKD: i nodi sono collegati da link fisici (linee continue) e virtuali (punteggiata) e ciascuno è controllato dal controller tramite il proprio SDN agent (tratteggio). Il nodo in primo piano mostra le componenti interne: i moduli QKD, esposti come interfacce, il key manager, l'interfaccia verso le applicazioni e l'agent. Adattata da [9, Fig. 1].

5. Applicazioni. Dal punto di vista del nodo, un'applicazione è qualunque entità che chiede chiavi al key manager.

Definizione 2.7 (Applicazione QKD)**.** Un'*applicazione QKD* (`qkd_app`) è un'istanza in esecuzione che consuma chiavi da un nodo SD-QKD. È *esterna* (`CLIENT`) se si tratta di un utente, un HSM, un cifratore, una funzione di rete virtuale o un protocollo di sicurezza; è *interna* (`INTERNAL`) se usa le chiavi per il funzionamento della rete stessa, come l'autenticazione o il relay di un link virtuale.

Ogni applicazione dichiara una qualità del servizio: banda massima e minima di chiave (`max_bandwidth`, `min_bandwidth`, in bit/s), `jitter` massimo della consegna in ms, tempo di vita `ttl` di una chiave inutilizzata nel key store e la possibilità di condividere cammini o chiavi tra più client. È la risposta tecnica all'esigenza di QoS differenziata del caso UC2.

6. Controller SDN. Al vertice c'è il controller (nella 015 anche *SD-QNC*, *Software-Defined QKD Network Controller*), che ha la vista globale del dominio e svolge tre ruoli. Fa da *registro centrale* delle applicazioni, così che un'applicazione debba conoscere solo il proprio key manager e non la topologia della rete. Pianifica e crea i *link*, fisici o virtuali, in base a domande e statistiche. Coordina infine il proprio dominio con il controller della rete ottica (*SD-ONC*) quando serve un nuovo cammino ottico per un canale quantistico.

Che cosa non sale al controller. L'informazione che attraversa il piano di controllo è volutamente minima: identificativi, stati, tassi, parametri fisici. Il materiale di chiave *non lascia mai il perimetro del nodo*. Per questo, osserva la specifica, introdurre SDN in una rete a nodi fidati non aggiunge rischi nuovi oltre a quelli già noti, come il denial of service.

2.2.3 Anatomia di un modello YANG

Sia la 015 sia la 018 sono scritte in YANG [25], un linguaggio nato in IETF per descrivere i dati di configurazione e di stato degli apparati di rete e le operazioni che si possono invocare su di essi. Un modello YANG non è codice eseguibile: è uno *schema*, come un XSD per XML o uno schema JSON, che stabilisce quali dati esistono, di che tipo sono, come sono annidati e chi può modificarli. Su quello schema poi lavorano i protocolli NETCONF e RESTCONF. Vediamo i costrutti principali, usando esempi tratti dai file della 015 [26].

Il modulo e la sua intestazione. L'unità di base è il *modulo*. L'intestazione dichiara la versione del linguaggio, un `namespace` globale univoco, un `prefix` breve con cui gli altri moduli lo citano, i moduli importati e le revisioni.

```
module etsi-qkd-sdn-node {
  yang-version "1";
  namespace "urn:etsi:qkd:yang:etsi-qkd-node";
  prefix "etsi-qkdn";
  import ietf-yang-types   { prefix "yang"; }
  import ietf-inet-types   { prefix "inet"; }
  import etsi-qkd-node-types { prefix "etsi-qkdn-types"; }
  organization "ETSI␣ISG␣QKD";
  revision "2022-01-30" { reference "ETSI␣GS␣QKD␣015␣V2.1.1"; }
  ...
}
```

La 015 è divisa in due moduli: `etsi-qkd-node-types` contiene solo tipi, `etsi-qkd-sdn-node` contiene il modello vero e proprio. È una scelta comune: separare i tipi permette ad altri moduli, come quelli della 018, di riusarli senza importare tutto il resto.

Tipi: built-in, typedef e identity. YANG offre tipi predefiniti (`boolean`, `string`, `uint32`, `decimal64`, `enumeration`…) e tipi comuni definiti dall'IETF, come `yang:uuid` o `yang:date-and-time`. Con `typedef` si definiscono tipi derivati con un nome proprio. Per valori simbolici estendibili si usano le `identity`: si dichiara un'identità base e poi identità che la derivano; un campo di tipo `identityref` accetta qualunque identità derivata dalla base. È così che la 015 definisce i tipi di link:

```
identity QKD-LINK-TYPES { description "QKD␣key␣association␣link␣types."; }
identity VIRT { base QKD-LINK-TYPES; description "Virtual␣Link."; }
identity PHYS { base QKD-LINK-TYPES; description "Physical␣Link."; }

typedef qkd-link-types {
  type identityref { base QKD-LINK-TYPES; }
}
```

Il vantaggio rispetto a un'`enumeration` è che un altro modulo può aggiungere, per esempio, un nuovo tipo di link senza modificare l'originale. Allo stesso modo sono definiti le tecnologie (`CV-QKD`, `DV-QKD`…), gli stati di nodo (`NEW`, `OPERATING`, `DOWN`, `FAILURE`, `OUT`), i ruoli (`TRANSMITTER`, `RECEIVER`, `TRANSCEIVER`) e i tipi di applicazione (`CLIENT`, `INTERNAL`).

Nodi dati: container, list, leaf, leaf-list. I dati sono organizzati ad albero con quattro costrutti:

- `leaf`: un singolo valore tipizzato (per esempio `qkdn_id`);
- `leaf-list`: una sequenza di valori dello stesso tipo (per esempio `virt_next_hop`, che ammette più successori);
- `container`: un nodo che raggruppa altri nodi, senza valore proprio (per esempio `qkdn_capabilities`);
- `list`: una collezione di elementi strutturati, ciascuno identificato da una `key` (per esempio la lista `qkd_link` con chiave `qkdl_id`).

Configurazione e stato. Ogni nodo è di *configurazione* (`config true`, il default: il client può scriverlo) oppure di *stato operativo* (`config false`: lo produce l'apparato e il client può solo leggerlo). Nella 015, per esempio, `qkdl_enable` è configurabile, mentre `qkdn_status`, `skr`

ed `eskr` sono stato. Questa distinzione riflette la divisione tra ciò che il controller *decide* e ciò che il nodo *misura*.

Riuso ed estensione: grouping, uses, augment. Un `grouping` è un blocco di nodi riutilizzabile, che viene copiato dove compare `uses`. Il modello della 015 è costruito quasi tutto così: `app_basic`, `app_details`, `app_qos`, `qkd_links` e così via sono grouping poi assemblati nel container `qkd_node`. L'`augment` fa l'operazione opposta: aggiunge nodi a un albero definito *in un altro modulo*, senza toccarlo. È lo strumento che la 018 userà per arricchire la propria topologia con i dati di inventario.

Riferimenti e vincoli. Un `leafref` è un campo il cui valore deve coincidere con un valore esistente altrove nell'albero, come una chiave esterna in un database. Le istruzioni `when` e `must` aggiungono condizioni, per esempio che un campo esista solo se il link è di un certo tipo. La 015 indica proprio così che i campi `virt_*` valgono solo per link virtuali e i campi `phys_*` solo per quelli fisici.

Operazioni e notifiche: rpc e notification. Oltre ai dati, un modulo può definire operazioni invocabili dal client (`rpc`, con `input` e `output`) e messaggi che il server invia spontaneamente (`notification`). La 015 definisce soprattutto notifiche; la 018, come vedremo, definisce anche RPC.

Leggere un albero. Lo strumento `pyang` produce una vista ad albero di un modulo, che useremo nel seguito. La notazione è compatta: `rw` indica un nodo di configurazione, `ro` un nodo di stato, `?` un nodo facoltativo, `*` una lista o leaf-list, la chiave di una lista compare tra parentesi quadre e `---n` indica una notifica.

Dai dati al messaggio. Lo stesso schema produce sia XML sia JSON. In JSON [27] il primo livello è qualificato con il nome del modulo e i valori di tipo `identityref` riportano il modulo che definisce l'identità. Un link fisico esposto da un nodo apparirebbe così:

```
{ "etsi-qkd-sdn-node:qkd_node": {
    "qkdn_id": "0f3c...-a1",
    "qkd_links": { "qkd_link": [ {
        "qkdl_id": "7b2e...-c4",
        "qkdl_type": "etsi-qkd-node-types:PHYS",
        "qkdl_local":  { "qkdn_id": "0f3c...-a1", "qkdi_id": 1 },
        "qkdl_remote": { "qkdn_id": "9d41...-e7", "qkdi_id": 2 },
        "phys_wavelength": 1550,
        "qkdl_performance": { "skr": 20000, "eskr": 12000,
                              "expected_consumption": 8000 } } ] } } }
```

(I valori sono illustrativi.)

2.2.4 Il modello del nodo SD-QKD

Con questi strumenti il modello della 015 si legge facilmente. Il container radice `qkd_node` ha quattro sezioni, una per ciascuna componente descritta sopra, più le notifiche:

```
module: etsi-qkd-sdn-node
  +--rw qkd_node
     +--rw qkdn_id              yang:uuid
     +--ro qkdn_status?         etsi-qkdn-types:qkdn-status-types
     +--rw qkdn_version?        string
     +--rw qkdn_location_id?    string
```

```
+--rw qkdn_capabilities
|  +--rw link_stats_support?          boolean
|  +--rw application_stats_support?   boolean
|  +--rw key_relay_mode_enable?       boolean
+--rw qkd_applications
|  +--rw qkd_app* [app_id]
|     +--rw app_id, app_type, server_app_id, client_app_id*, ...
|     +--rw app_qos            (max/min_bandwidth, jitter, ttl, ...)
|     +--rw backing_qkdl_id*, local_qkdn_id, remote_qkdn_id
+--rw qkd_interfaces
|  +--rw qkd_interface* [qkdi_id]
|     +--rw qkdi_id, qkdi_model, qkdi_type
|     +--rw qkdi_att_point     (device, port)
|     +--rw qkdi_capabilities  (role, wavelength_range, max_absorption)
|     +--ro qkdi_status
+--rw qkd_links
   +--rw qkd_link* [qkdl_id]
      +--rw qkdl_id, qkdl_status, qkdl_enable, qkdl_type
      +--rw qkdl_local / qkdl_remote   (qkdn_id, qkdi_id)
      +--rw qkdl_applications*
      +--rw virt_prev_hop, virt_next_hop*, virt_bandwidth   (VIRT)
      +--rw phys_channel_att, phys_wavelength, phys_qkd_role (PHYS)
      +--rw qkdl_performance   (expected_consumption, skr, eskr,
                                phys_perf* [perf_type]: QBER, SNR)
```

Identità del nodo. `qkdn_id` è un UUID univoco nella rete; `qkdn_status`, `qkdn_version` e `qkdn_location_id` servono a inventario e diagnostica.

Capability. Le tre capability dicono al controller che cosa il nodo è disposto a fare: esporre statistiche sui link (SKR, consumo, stato, QBER), esporre statistiche sulle applicazioni e fare da relay. Un nodo con `key_relay_mode_enable = false` non verrà mai scelto come nodo intermedio di un link virtuale.

Applicazioni. Ogni `qkd_app` lega un'applicazione al servizio di chiave: `server_app_id` identifica l'entità che ha avviato la richiesta, `client_app_id` le entità autorizzate a ricevere la stessa chiave, `backing_qkdl_id` il link che fornisce le chiavi, `local_qkdn_id` e `remote_qkdn_id` i due nodi coinvolti. Finché il nodo remoto non è noto, il nodo locale non può servire chiavi. Le statistiche (`app_statistics`) riportano i bit consumati per intervallo di tempo.

Interfacce. Ogni `qkd_interface` descrive un modulo: modello, tecnologia, stato, punto di attacco a un dispositivo ottico (`qkdi_att_point`) e capability fisiche. È l'informazione che serve al controller per abbinare due interfacce compatibili e chiedere alla rete ottica un cammino tra le rispettive porte.

Link e prestazioni. Ogni `qkd_link` riporta gli estremi, il tipo, lo stato, le applicazioni servite e i parametri specifici del tipo. Il blocco `qkdl_performance` contiene le tre grandezze che guidano tutta la pianificazione:

- `skr`: tasso di generazione di chiave segreta, in bit/s;
- `expected_consumption`: somma delle bande di tutte le applicazioni sul link;
- `eskr`: tasso *effettivo*, cioè disponibile dopo il consumo interno.

In forma compatta, se $\mathcal{A}_\ell$ è l'insieme delle applicazioni sul link ℓ, ciascuna con banda b_a,

$$\text{expected_consumption}_\ell = \sum_{a \in \mathcal{A}_\ell} b_a, \qquad \text{ESKR}_\ell \approx \text{SKR}_\ell - \text{expected_consumption}_\ell. \tag{2}$$

La seconda relazione è una lettura intuitiva dei campi, non una formula dello standard. Per i link fisici si aggiungono metriche come QBER e SNR.

2.2.5 Notifiche

Interrogare continuamente i nodi sarebbe inefficiente. La 015 prevede quindi notifiche YANG dal nodo al controller, divise per oggetto:

- **applicazioni**: `sdqkdn_application_new` (una nuova applicazione chiede chiavi), `sdqkdn_application_qos_update`, `sdqkdn_application_disconnected`;
- **interfacce**: `sdqkdn_interface_new`, `sdqkdn_interface_down` (con un campo *reason*), `sdqkdn_interface_out`;
- **link**: `sdqkdn_link_down`, `sdqkdn_link_perf_update` (variazione significativa delle prestazioni), `sdqkdn_link_overloaded` (il link non regge la domanda).

A queste si aggiunge una struttura generica di eventi e allarmi, pensata per accogliere nuovi tipi di evento senza cambiare il modello. Inoltre i protocolli basati su YANG permettono di sottoscrivere qualunque dato del datastore, così che ogni variazione possa essere segnalata in tempo reale.

2.2.6 I workflow

La clausola 5 descrive tre scenari fondamentali, che mostrano le componenti all'opera.

Registrazione di un'applicazione. È il caso più istruttivo, perché spiega perché il controller fa da registro centrale (Fig. 3). Un'applicazione conosce solo l'indirizzo del proprio key manager; non conosce il nodo dell'applicazione con cui vuole comunicare.

1. L'applicazione nel dominio di sicurezza A si collega al proprio nodo tramite la key delivery API, inviando il proprio ID, l'ID dell'applicazione remota e la QoS richiesta.
2. Il nodo A non sa dove si trovi l'applicazione remota e inoltra la richiesta al controller, aggiungendo il proprio ID.
3. Se l'applicazione remota non è ancora registrata, il controller risponde con un semplice riscontro.
4. Nel dominio B l'applicazione remota si collega al proprio nodo, che a sua volta informa il controller.
5. Il controller riconosce la coppia, genera un identificativo unico di servizio (`app_id`) e invia a entrambi i nodi tutte le informazioni necessarie: ID di applicazioni e nodi, `app_id` e QoS.
6. I nodi comunicano l'`app_id` alle applicazioni e riservano chiavi per esse; da qui in poi le applicazioni chiedono chiavi usando quell'identificativo.

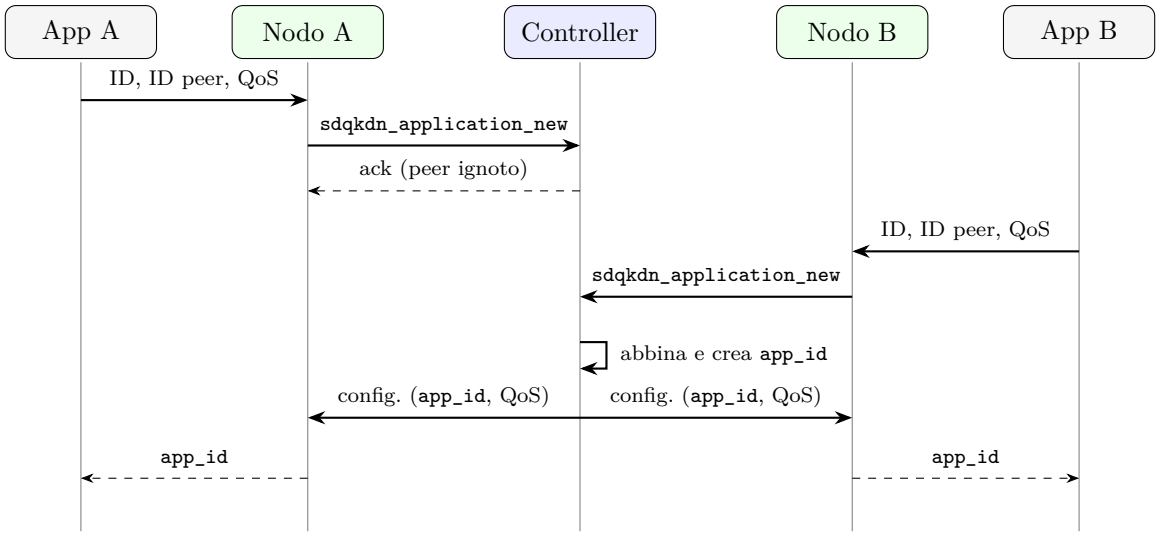


Figura 3: Registrazione di un'applicazione in una rete SD-QKD. Le applicazioni conoscono solo il proprio key manager; è il controller ad abbinare le due richieste e a creare l'identificativo di servizio. Adattata da [9, Fig. 2].

Creazione di un link fisico. Il controller riceve un innesco, da una domanda rilevata sui nodi o dalla propria NBI. Cerca nei due siti due interfacce libere e compatibili, chiede al controller della rete ottica (SD-ONC) un cammino ottico (*lightpath*) con i vincoli del canale quantistico e, ricevuta la conferma, configura le due interfacce sui rispettivi nodi. Se i due controller sono integrati in uno solo, l'operazione è interna. Questo workflow è il cuore della coesistenza tra QKD e rete ottica chiesta dal caso UC4.

Creazione di un link virtuale. Se non ci sono interfacce compatibili o non esiste un lightpath adatto, il controller ripiega su un link virtuale. Calcola un cammino sulla rete QKD esistente, per esempio su un grafo pesato costruito con disponibilità di chiave, tassi di generazione e consumi; configura ogni nodo intermedio in modalità relay indicando precedente e successivo; infine configura gli estremi con la banda da fornire. La specifica non prescrive l'algoritmo. Una scelta naturale, coerente con la (2), è pesare ciascun link con l'inverso della chiave disponibile e scartare i link che non reggono la banda r richiesta:

$$w_\ell = \frac{1}{\mathrm{ESKR}_\ell}, \qquad p^\star = \arg \min_{p\,:\,\min_{\ell\in p}\mathrm{ESKR}_\ell \geq r} \sum_{\ell\in p} w_\ell. \tag{3}$$

La (3) è un esempio nostro, non parte dello standard. La specifica avverte che, se gestiti male, i link virtuali possono congestionare i link fisici sottostanti: è proprio per questo che serve un controller con la vista globale di consumi e disponibilità.

2.2.7 Protocolli e sicurezza

La 015 non impone un protocollo, ma indica i due più diffusi tra quelli basati su YANG: NETCONF [28], tipicamente tra apparato e controller, transazionale e trasportato su SSH o TLS con codifica XML; e RESTCONF [29], tipicamente sulla NBI dei controller, basato su HTTP con operazioni CRUD e codifica JSON o XML.

Sul piano della sicurezza (clausola 6) la specifica distingue due casi all'installazione di un nuovo sistema. Se il sistema entra in un'area sicura già presente nella rete, il canale verso il controller può essere protetto con chiavi QKD prodotte da link esistenti. Se invece si aggiunge un nuovo luogo sicuro, serve uno schema iniziale (un'identità fidata o un segreto pre-condiviso,

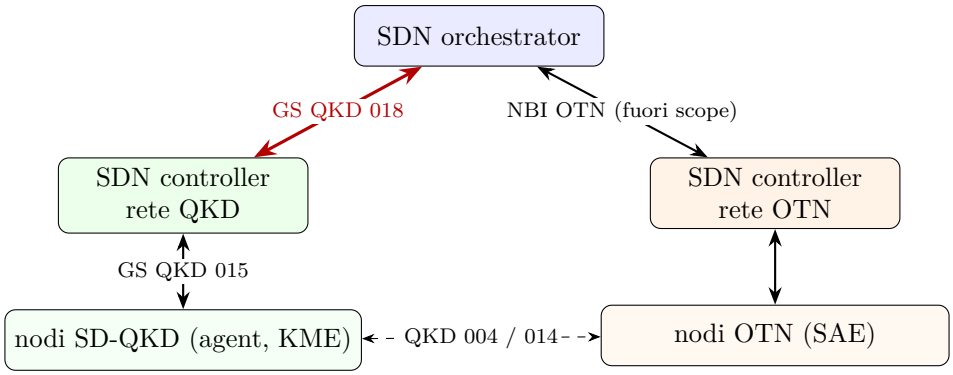


Figura 4: Scenario di riferimento della GS QKD 018 (adattato da [10, Fig. 1]). In rosso l'interfaccia standardizzata; la linea tratteggiata è la consegna delle chiavi dai KME ai SAE, che avviene nello stesso perimetro di sicurezza.

usato solo durante l'installazione) per presentare il nuovo SDN agent al controller; dopo la prima autenticazione si passa a chiavi QKD. È lo stesso principio di key growing visto in apertura.

2.2.8 Dal dominio singolo all'orchestrazione

La 015 risolve il controllo di *un* dominio QKD da parte di *un* controller. Restano fuori tre problemi che la GS QKD 002 aveva già fatto intravedere. Primo, un operatore può avere più domini QKD, magari di vendor diversi, ciascuno con il proprio controller. Secondo, le applicazioni che consumano chiave possono stare in un'altra rete, per esempio nei nodi della rete di trasporto ottico, controllata da un controller diverso. Terzo, l'operatore vuole gestire la QKD con gli stessi strumenti di orchestrazione che usa per il resto della rete. Serve quindi un livello superiore e un'interfaccia standard per raggiungerlo: è il compito della GS QKD 018.

3 ETSI GS QKD 018

La GS QKD 018 (V1.1.1, aprile 2022) [10] definisce l'*orchestration interface* tra un SDN orchestrator e l'SDN controller di una rete QKD. Il controller agisce da *server*, l'orchestrator da *client*; il modello informativo è espresso in YANG ed è indipendente dal vendor. Nella terminologia SDN è la *northbound interface* del controller QKD, cioè l'esatto complemento della GS QKD 015. La gerarchia complessiva diventa:

$$\text{orchestrator} \xrightarrow{\text{QKD 018}} \text{controller} \xrightarrow{\text{QKD 015}} \text{SDN agent / nodo SD-QKD} \xrightarrow{\text{QKD 004/014}} \text{SAE}.$$

Definizione 3.1 (Orchestrazione SDN)**.** L'*orchestrazione SDN* è il processo continuo con cui le risorse disponibili vengono coordinate automaticamente, secondo criteri di ottimizzazione, per stabilire e rilasciare servizi end-to-end che attraversano più domini di rete, ciascuno governato dal proprio controller SDN [10, §4.2].

Definizione 3.2 (Service link)**.** Un *service link* è un collegamento logico di associazione di chiave tra due nodi QKD, realizzato da uno o più link fisici QKD (single-hop o multi-hop). A livello di orchestrazione corrisponde al link virtuale della GS QKD 015.

3.1 Use case della QKD 018

Il caso d'uso da cui nasce lo standard (§4.1) riprende quasi alla lettera il problema del backbone della GS QKD 002. Un operatore che introduce la QKD può far convivere canali quantistici

e classici nella stessa fibra oppure tenerli separati. La convivenza è difficile per due motivi: le potenze ottiche dei canali classici sono di molti ordini di grandezza superiori e disturbano il canale quantistico, e le distanze raggiungibili dai due tipi di canale sono molto diverse. In un ambiente ricco di fibra l'operatore può quindi scegliere una rete QKD separata dalla rete di trasporto ottico (OTN), con vantaggi di prestazioni, isolamento e separazione delle responsabilità. Seguendo il paradigma SDN, le due reti sono governate da controller diversi.

Nasce così un problema di coordinamento: i SAE che devono cifrare il traffico stanno nei nodi OTN, le chiavi sono generate nei nodi QKD e ciascun controller conosce solo la propria rete. L'orchestrator risolve tre questioni:

1. **Address matching**: associa l'indirizzo del nodo OTN che deve ricevere chiavi a quello del nodo QKD che le fornisce; i due si trovano nello stesso perimetro di sicurezza (Fig. 4);
2. **provisioning end-to-end**: traduce la richiesta di un cliente nella configurazione dei singoli domini, sceglie i domini da attraversare e coordina i cammini;
3. **gestione**: raccoglie topologia, inventario delle risorse di chiave e informazioni FCAPS di ogni dominio.

Generalizzando, lo stesso schema permette di orchestrare *più domini QKD di vendor diversi*, ciascuno dietro il proprio controller, oltre a domini QKD e classici insieme. Lo standard sottolinea anche un vantaggio pratico: un operatore che ha già un orchestrator per le proprie reti ottiche può integrare la rete QKD senza introdurre strumenti di gestione separati, che è esattamente il requisito di gestione integrata del caso UC4.

Le funzioni richieste all'orchestrator (§4.2) sono:

- tradurre le richieste di servizio dei clienti nella configurazione dei domini e nell'allocazione dei SAE;
- richiedere attivazione e rilascio dei servizi, incluse le connessioni tra domini;
- calcolare cammini multi-dominio, con le connessioni inter-dominio e gli estremi;
- modificare i cammini su vincoli dell'operatore;
- scoprire nodi e link di ciascun controller e mantenerne una vista astratta;
- monitorare le risorse di chiave disponibili su nodi e link;
- monitorare la gestione FCAPS e riceverne le notifiche.

3.2 Il modello informativo

La clausola 5 traduce queste funzioni in cinque blocchi informativi.

Discovery della topologia (§5.1). Il controller espone nodi SD-QKD, interfacce e link fisici, e in seguito i service link. L'orchestrator non ha bisogno della composizione fisica completa, ma di una vista *astratta* di ciascun dominio e della connettività tra domini. La discovery può essere *proattiva* (l'orchestrator interroga il controller prima di ogni nuovo calcolo di cammino) o *reattiva* (il controller notifica ogni cambiamento, per esempio l'aggiunta o la rimozione di un nodo).

Stato della rete e inventario (§5.2). Per ogni nodo e link fisico il controller espone lo stato, le prestazioni fisiche e soprattutto le risorse di chiave: il tasso di generazione e il tasso disponibile dopo il consumo interno. Per i service link si aggiungono la banda, la chiave disponibile e la politica di ripristino in caso di guasto.

Stato dei servizi end-to-end (§5.3). L'orchestrator può interrogare le applicazioni esterne attive, i nodi locale e remoto e i service link che le servono, con il relativo stato e tasso di chiave.

Provisioning con calcolo dei cammini (§5.4). L'orchestrator chiede al controller cammini che soddisfino certi vincoli; il controller restituisce una lista di candidati, eventualmente più volte con vincoli diversi, finché l'orchestrator non ne sceglie uno da attivare. In caso di guasto l'orchestrator può chiedere di sostituire il cammino.

Notifiche (§5.5). Il controller segnala eventi, allarmi e superamenti di soglia. Le soglie sulle prestazioni (QBER, SKR, ESKR) sono configurabili dall'orchestrator. Rispetto alla 015 si aggiungono notifiche sullo stato dei nodi (`sdqkdn_new`, `_operating`, `_down`, `_failure`, `_out`), sulle applicazioni attivate (`sdqkdn_application_deployed`) e sui service link (`sdqkdn_svc_link_down`, `_perf_update`, `_overloaded`).

La (2) si estende in modo naturale al livello di orchestrazione. Poiché in un cammino multi-hop p ogni link inoltra la stessa chiave, un nuovo servizio con richiesta r è sostenibile solo se il collo di bottiglia lo regge:

$$p \text{ ammissibile} \iff \min_{\ell \in p} \mathrm{ESKR}_\ell \;\geq\; r. \tag{4}$$

Anche questa è una formalizzazione nostra, coerente con i campi `skr`, `eskr` ed `expected_consumption` dei modelli YANG; lo standard non prescrive alcun algoritmo di calcolo dei cammini.

3.3 I moduli YANG

L'Annex A (normativo) traduce il modello in tre moduli YANG 1.1, pubblicati su ETSI Forge [18]. Con i costrutti visti in §2.2.3 la loro struttura si legge rapidamente. I moduli importano `etsi-qkd-sdn-node` ed `etsi-qkd-node-types` della GS QKD 015 per riusarne i tipi (identificativi, tipi di link, QoS delle applicazioni): la 018 non reinventa il nodo, ne fornisce una vista astratta. I tre moduli formano una catena: *topology* (lo scheletro) → *inventory* (che cosa c'è e in che stato) → *connectivity* (che cosa ci si fa).

`etsi-qkd-sdn-topology` (prefisso `qkdnw-tplg`). Descrive solo la struttura, cioè chi è collegato a chi:

```
qkd_network
 +-- sdqkd_nodes
 |    +-- qkdn* [qkdn_id]                          nodi SD-QKD
 |         +-- qkd_interfaces/qkdi* [qkdi_id]      moduli QKD
 +-- qkd_links
      +-- qkd_phys_links/phys_link* [phys_link_id]  link fisici
      |    +-- link_type
      |    +-- phys_link_local_qkdn   (qkdn_id_ref, qkdi_id_ref)
      |    +-- phys_link_remote_qkdn  (qkdn_id_ref, qkdi_id_ref)
      +-- qkd_svc_links/svc_link* [svc_link_id]     service link
           +-- ... estremi locale/remoto come sopra
```

Rispetto alla 015 il punto di vista cambia: non c'è più un albero per nodo, ma un unico albero per l'intera rete del dominio. In termini astratti è un grafo $G = (V, E)$ con V insieme dei nodi `qkdn` ed $E = E_{\text{phys}} \cup E_{\text{svc}}$, dove i `phys_link` rappresentano adiacenze fisiche e i `svc_link` relazioni logiche di chiave, anche multi-hop. Gli estremi sono `leafref`, cioè riferimenti validati a nodi e interfacce esistenti. Avendo il grafo G_i di ciascun dominio i, l'orchestrator compone

una vista multi-dominio

$$G = \Big(\bigcup_i V_i,\ \bigcup_i E_i \cup E_{\text{inter}} \Big), \tag{5}$$

dove E_{inter} sono le adiacenze tra domini, tipicamente realizzate da nodi di bordo condivisi.

`etsi-qkd-sdn-inventory` (prefisso `qkdnw-invt`). Non crea un nuovo albero ma fa `augment` di quello di topologia, aggiungendo foglie agli elementi già definiti (Tab. 2). È un esempio da manuale dell'uso di `augment`: chi è interessato solo alla struttura implementa la topologia, chi vuole anche lo stato aggiunge l'inventario senza duplicare nulla.

Tabella 2: Elementi aggiunti dal modulo inventory all'albero di topologia.

Elemento	Foglie aggiunte
`qkdn`	versione, capability, location, indirizzo IP e porta del KME (dove i SAE prelevano le chiavi via QKD 014), stato
`qkdi`	ruolo supportato, modello, tipo, punto di attacco, stato
`phys_link`	`enable` (configurabile), `wavelength`, `performance` (`skr`, `eskr`, `expected_consumption`, QBER, SNR), stato
`svc_link`	`bandwidth`, `performance`, `path_restoration` (`AUTOMATIC`/`MANUAL`, `hold_off_time` in ms), stato

Il modulo definisce anche le notifiche che realizzano la parte reattiva della §5.5: `sdqkdn_new`, `sdqkdn_interface_new`, `sdqkdn_phys_link_new`, `sdqkdn_svc_link_new`, le varianti `_perf_update` e `_overloaded` e le generiche `alarm` ed `event`. Il campo `eskr` dei link è il dato da cui l'orchestrator capisce se un cammino regge un nuovo servizio, secondo la (4).

`etsi-qkd-sdn-connectivity` (prefisso `qkdnw-conn`). È il modulo operativo: gestisce le applicazioni e i cammini che le servono.

```
connectivity_service/qkd_service/qkd_app* [app_id]
 +-- (dati dell'applicazione da QKD 015: stato, tipo, QoS, statistiche)
 +-- app_mapped_svc_link
      +-- svc_link_and_qkdn_ids   service link ed estremi
      +-- path_constraints        include_nodes / exclude_nodes
      +-- candidate_paths         num_of_candidate_paths + lista di path
      +-- deploy_path             path_id scelto
```

Qui compaiono le `rpc`, che ricalcano passo per passo il workflow di provisioning:

`app-registration`
: registra l'applicazione nel dominio;

`create-service-link-id-for-app`
: crea il service link tra nodo locale e remoto;

`create-candidate-paths-per-service-link-for-app`
: il controller calcola i cammini candidati rispettando QoS e vincoli `include_nodes`/`exclude_nodes`;

`update-candidate-paths-per-service-link-for-app`
: ricalcolo con vincoli diversi;

`deploy-a-candidate-path-per-service-link-for-app`
: l'orchestrator sceglie un `path_id` e il controller lo attiva;

`delete-deployed-path-per-service-link-for-app`
: rilascio del cammino.

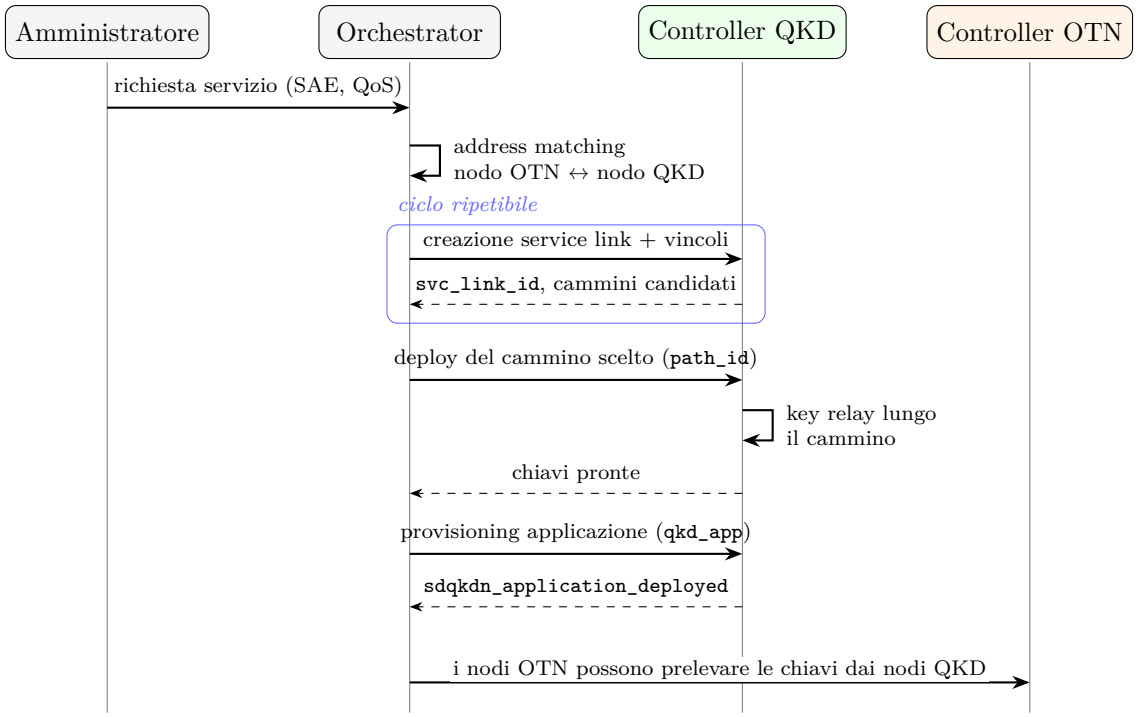


Figura 5: Provisioning end-to-end di un servizio QKD tra rete QKD e OTN (rielaborazione semplificata di [10, Fig. 2]). L'interazione con il controller OTN è fuori dallo scope della specifica.

Le notifiche del modulo sono `sdqkdn_application_new`, `sdqkdn_application_deployed`, `sdqkdn_application_qos_update` e `sdqkdn_application_disconnected`.

Il punto concettuale è la **divisione dei ruoli**: il controller conosce il proprio dominio e *propone* i cammini, l'orchestrator ha la vista d'insieme e *decide*. La Tab. 3 riassume.

Tabella 3: I tre moduli YANG della GS QKD 018.

Modulo	Domanda	Clausola	Dati
topology	Com'è fatta la rete?	§5.1	struttura (grafo)
inventory	Quante risorse ci sono e in che stato?	§5.2, §5.5	stato, prestazioni, configurazione, notifiche
connectivity	Come servo un'applicazione end-to-end?	§5.3, §5.4, §6.2	applicazioni, RPC di calcolo e deploy dei cammini

3.4 Workflow di provisioning multi-dominio

La clausola 6 lega le informazioni al ciclo di vita del servizio. Quando la rete QKD viene introdotta, l'orchestrator ne acquisisce la topologia (§5.1); durante l'esercizio riceve, su richiesta o periodicamente, stato e inventario (§5.2); dopo l'attivazione di un servizio ne monitora lo stato (§5.3); in ogni momento riceve notifiche (§5.5).

La §6.2 descrive il caso completo (Fig. 5): un orchestrator coordina il controller QKD e il controller OTN, che non comunicano tra loro direttamente.

I passi sono:

1. l'amministratore chiede all'orchestrator un servizio, indicando i SAE nei nodi OTN e la loro QoS;

2. l'orchestrator individua i controller coinvolti ed effettua l'address matching; conosce già gli indirizzi dei nodi QKD grazie a topologia e inventario;
3. chiede al controller QKD la creazione di un service link e il calcolo dei cammini, con i vincoli desiderati;
4. il controller assegna un `svc_link_id` e restituisce i cammini candidati, con una priorità per ciascuno; il passo può ripetersi con vincoli diversi;
5. l'orchestrator sceglie un cammino e ne chiede il deploy; i nodi generano e inoltrano chiavi lungo il cammino finché i due estremi sono pronti;
6. ricevuta la conferma, l'orchestrator richiede il provisioning dell'applicazione sul service link e avvisa il controller OTN che i nodi OTN possono iniziare a ricevere chiavi dai nodi QKD indicati.

Si noti il parallelo con il workflow del link virtuale della 015: lì il controller calcolava e configurava il cammino da solo, qui lo *propone* e attende la scelta dell'orchestrator. Con più domini QKD l'orchestrator ripete i passi 3–5 per ciascun dominio lungo il cammino inter-dominio scelto, collegando i segmenti attraverso i nodi di bordo: è ciò che fanno Sim et al. (§5.1).

3.5 Protocolli e sicurezza

Trasporto (§8). Come la 015, lo standard separa il modello dei dati (YANG 1.1) dal protocollo di trasporto, che può essere NETCONF [28] o RESTCONF [29]. Con RESTCONF ogni risorsa è un URI, la lettura usa `GET` e la configurazione `POST`, `PUT`, `PATCH` e `DELETE`, con codifica XML o JSON; le notifiche si ricevono sottoscrivendo i relativi stream. Lo standard osserva che il paradigma REST, senza stato, si presta bene a questo tipo di API. A titolo di esempio, la lettura della topologia e l'invocazione di una RPC avrebbero la forma:

```
GET  /restconf/data/etsi-qkd-sdn-topology:qkd_network
POST /restconf/operations/etsi-qkd-sdn-connectivity:create-service-link-id-for-app
```

Sicurezza (§7). Il controller ha una vista centrale della rete e ne passa un sottoinsieme all'orchestrator; le misure di protezione dell'interfaccia (autenticazione, cifratura del canale, per esempio TLS sotto RESTCONF) sono lasciate all'implementazione. Il principio fondamentale, ereditato dalla GS QKD 015, è che **le chiavi derivate da QKD non transitano mai né per il controller né per l'orchestrator**: nei moduli YANG compaiono solo tassi di chiave e l'indirizzo del KME. Un orchestrator compromesso può quindi causare disservizi (cammini sbagliati, risorse esaurite, denial of service) ma non leggere le chiavi. Lo standard raccomanda procedure accurate di progettazione, installazione e personalizzazione per mitigare richieste malevole e attacchi ai canali di controllo.

4 L'analogia con ACTN (RFC 8453)

La GS QKD 018 non inventa un modello nuovo: applica alla QKD un pattern già standard nelle reti di trasporto, il framework ACTN (*Abstraction and Control of TE Networks*) dell'IETF [30]. ACTN prevede tre livelli: il *Customer Network Controller* (CNC), il *Multi-Domain Service Coordinator* (MDSC) e i *Provisioning Network Controller* (PNC) di ciascun dominio, collegati dalle interfacce CMI (CNC–MDSC) e MPI (MDSC–PNC). La corrispondenza con la GS QKD 018 è diretta (Tab. 4).

La differenza sostanziale sta nella risorsa. In ACTN la banda di un link è fissata dalla capacità del canale; in QKD l'SKR fluttua con le condizioni fisiche e la chiave si accumula in

Tabella 4: Corrispondenza tra ACTN e la pila ETSI per la QKD.

ACTN	ETSI QKD	Osservazione
CNC	amministratore / portale cliente	origina la richiesta di servizio
MDSC	SDN orchestrator	vista multi-dominio, decisione del cammino
PNC	SDN controller del dominio QKD	calcola i candidati, attiva i cammini
MPI	interfaccia GS QKD 018	topologia astratta, provisioning, notifiche
SBI	interfaccia GS QKD 015	controllo dei singoli nodi
TE topology astratta	`etsi-qkd-sdn-topology`	vista vendor-neutral
banda TE	SKR / ESKR	la risorsa da allocare è la chiave

buffer finiti, da cui viene consumata. Per questo nella 018 notifiche come `_perf_update` e `_overloaded` e la politica di `path_restoration` hanno un ruolo più centrale che nelle reti di trasporto classiche.

5 Casi d'uso reali

5.1 SK Telecom: orchestrazione multi-vendor su KOREN

Sim et al. [19] descrivono il primo dispiegamento commerciale documentato di un orchestrator conforme alla GS QKD 018, sul testbed della rete di ricerca coreana KOREN (2020–2022). Due operatori QKD gestiscono due reti con dispositivi e KMS di vendor diversi, ciascuna con il proprio controller (QKDNC); un orchestrator (QKDNO) le coordina. I link usano BB84 con decoy state su tratte tra 39 e 100 km. Le due reti si toccano in *nodi di bordo* (per esempio Pangyo e Daejeon) che non contengono moduli QKD ma due KMS, uno per operatore, interconnessi.

Quando due SAE appartengono a reti diverse, l'orchestrator calcola il cammino end-to-end con un *path computation engine*, scegliendo *a priori* il nodo di bordo da attraversare; ogni controller crea poi il proprio segmento di service link. Per legare i segmenti gli autori estendono il modello: introducono un tipo di applicazione *multi-segment* (`MS`), con un identificativo `ms_app_id` comune ai domini e le informazioni `server_ms_info` e `client_ms_info` sui KMS esterni, e una RPC `app-registration-ms`. Il trasferimento delle chiavi al nodo di bordo usa l'interfaccia KMS–KMS della GS QKD 020 [11]. Il caso mostra sia la validità dello schema 018 sia un suo limite: i tipi di applicazione ereditati dalla 015 (`CLIENT` e `INTERNAL`) non bastano per il multi-dominio, e servono estensioni. È un buon esempio dell'utilità delle `identity` YANG discusse in §2.2.3: un nuovo tipo si aggiunge senza riscrivere il modello di partenza.

5.2 Politecnico di Milano: controllo gerarchico con ONOS

La tesi di Vicario [20] realizza un prototipo end-to-end di controllo gerarchico per reti metropolitane: i controller di dominio sono basati su ONOS e gestiscono i nodi tramite il modello GS QKD 015, mentre l'interfaccia verso l'alto è la GS QKD 018 esposta via RESTCONF a un orchestrator scritto in Python che aggrega la topologia di più domini. È un'implementazione utile perché mostra l'intera pila 015 $\rightarrow$ 018 in software aperto, con gli stessi moduli YANG descritti in §3.3.

5.3 MadQCI: la rete di Madrid

MadQCI [21] è una rete QKD reale a Madrid con decine di moduli di vendor diversi su oltre 130 km, integrata con la rete ottica di produzione. Il controllo usa uno stack SD-QKD basato sulla GS QKD 015, che si conferma lo standard più adottato. La GS QKD 018 è citata come

strumento per l'orchestrazione inter-dominio e la coordinazione ottico-quantistica, ma gli autori la descrivono come un'attività ancora in corso. È una testimonianza onesta dello stato dell'arte: il controllo di dominio è maturo, l'orchestrazione molto meno.

6 Limiti e prospettive

- **Adozione.** La 015 è usata in reti reali, la 018 quasi solo in testbed e prototipi [21].
- **Multi-dominio incompleto.** Lo scenario normativo è QKD+OTN; il caso QKD+QKD richiede estensioni come quelle di Sim et al. e il coordinamento con la GS QKD 020 per il passaggio delle chiavi tra KMS.
- **Metriche.** L'ESKR è un indicatore istantaneo e non tiene conto della chiave già accumulata nei buffer né della sua dinamica. Laschet et al. [23] propongono indicatori più ricchi per la selezione dei cammini.
- **Riservatezza della topologia.** Esporre la topologia interna all'orchestrator è accettabile dentro un operatore, meno tra operatori diversi; servono schemi di instradamento inter-dominio che preservino la riservatezza [23, 24].
- **Sicurezza dell'interfaccia.** Lo standard non specifica i meccanismi di protezione del canale orchestrator–controller, che restano a carico dell'implementazione.
- **Algoritmi.** Il calcolo dei cammini non è standardizzato: è un vantaggio per l'innovazione [5, 22], ma rende meno prevedibile il comportamento in scenari multi-vendor.

7 Conclusioni

Letta in sequenza, la produzione ETSI sul controllo della QKD ha una logica chiara. La GS QKD 002 ha mostrato che la QKD utile è una rete, con gestione delle chiavi, requisiti di servizio differenziati e integrazione nella rete ottica dell'operatore. La GS QKD 015 ha risposto con l'astrazione del nodo SD-QKD: moduli come interfacce, un key manager unico, link fisici e virtuali, applicazioni con QoS, un controller che registra, pianifica e configura, tutto descritto in YANG. La GS QKD 018 ha aggiunto il livello di orchestrazione: ogni dominio diventa un grafo astratto, le risorse di chiave vengono monitorate tramite l'inventario e il provisioning segue uno schema in cui il controller propone e l'orchestrator decide. Il disegno ricalca ACTN, riusa YANG, NETCONF e RESTCONF e mantiene il principio di sicurezza essenziale: le chiavi non lasciano mai i nodi. Le esperienze di KOREN, del Politecnico di Milano e di MadQCI mostrano che lo schema funziona ma che l'orchestrazione QKD è ancora giovane. Il passo successivo è estenderla davvero al multi-operatore, con metriche migliori e garanzie di riservatezza tra domini.

Bibliografia

[1] Stefano Pirandola et al. «Advances in quantum cryptography». In: *Advances in Optics and Photonics* 12.4 (2020).

[2] Miralem Mehic et al. «Quantum Key Distribution: A Networking Perspective». In: *ACM Computing Surveys* 53.5 (2020).

[3] Yuan Cao, Yongli Zhao, Qin Wang, Jie Zhang, Soon Xin Ng e Lajos Hanzo. «The Evolution of Quantum Key Distribution Networks: On the Road to the Qinternet». In: *IEEE Communications Surveys & Tutorials* (2022).

[4] GÉANT GN5-1. *White Paper: QKD Concepts and Considerations*. Rapp. tecn. GÉANT, 2024. URL: https://resources.geant.org/wp-content/uploads/2024/02/GN5-1_White-Paper_QKD-Concepts-and-Considerations.pdf.

[5] *QKD Routing Protocol in Quantum Networks: Overview and Challenges*. arXiv:2407.13156. 2024.

[6] ETSI ISG QKD. *Quantum Key Distribution (QKD); Use Cases*. Group Specification GS QKD 002 V1.1.1. ETSI, giu. 2010. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/002/01.01.01_60/gs_QKD002v010101p.pdf.

[7] ETSI ISG QKD. *Quantum Key Distribution (QKD); Application Interface*. Group Specification GS QKD 004 V2.1.1. ETSI, 2020. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/004/02.01.01_60/gs_QKD004v020101p.pdf.

[8] ETSI ISG QKD. *Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API*. Group Specification GS QKD 014 V1.1.1. ETSI, 2019.

[9] ETSI ISG QKD. *Quantum Key Distribution (QKD); Control Interface for Software Defined Networks*. Group Specification GS QKD 015 V2.1.1. ETSI, apr. 2022. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/015/02.01.01_60/gs_QKD015v020101p.pdf.

[10] ETSI ISG QKD. *Quantum Key Distribution (QKD); Orchestration Interface for Software Defined Networks*. Group Specification GS QKD 018 V1.1.1. ETSI, apr. 2022. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/018/01.01.01_60/gs_qkd018v010101p.pdf.

[11] ETSI ISG QKD. *Quantum Key Distribution (QKD); Interoperable Key Management System*. Group Specification GS QKD 020. ETSI. URL: https://forge.etsi.org/rep/qkd/gs020-interop-kms.

[12] ITU-T. *Overview on networks supporting quantum key distribution*. Recommendation Y.3800. International Telecommunication Union, 2019.

[13] ITU-T. *Quantum key distribution networks – Control and management*. Recommendation Y.3804. International Telecommunication Union, 2025. URL: https://www.itu.int/dms_pubrec/itu-t/rec/y/T-REC-Y.3804-202504-I!!SUM-HTM-E.htm.

[14] ITU-T. *Quantum key distribution networks – Software-defined networking control*. Recommendation Y.3805. International Telecommunication Union, 2021.

[15] Diego Kreutz, Fernando M. V. Ramos, Paulo Veríssimo, Christian Esteve Rothenberg, Siamak Azodolmolky e Steve Uhlig. «Software-Defined Networking: A Comprehensive Survey». In: *Proceedings of the IEEE* 103.1 (2015).

[16] Alejandro Aguado et al. «First experimental demonstration of secure NFV orchestration over an SDN-controlled optical network with time-shared QKD resources». In: *European Conference on Optical Communication (ECOC)*. arXiv:1604.05861. 2016.

[17] Alejandro Aguado, Victor López, Juan Pedro Brito, Antonio Pastor, Diego R. López e Vicente Martín. «Enabling Quantum Key Distribution Networks via Software-Defined Networking». In: *International Conference on Optical Network Design and Modeling (ONDM)*. 2020. DOI: 10.23919/ONDM48393.2020.9133024.

[18] ETSI ISG QKD. *gs018-orch-int-sdn: YANG modules etsi-qkd-sdn-topology, etsi-qkd-sdn-inventory, etsi-qkd-sdn-connectivity (rev. 2022-02-22)*. ETSI Forge. 2022. URL: https://forge.etsi.org/rep/qkd/gs018-orch-int-sdn.

[19] Dong-Hi Sim, Jongyoon Shin e Min Hyung Kim. «Software-Defined Networking Orchestration for Interoperable Key Management of Quantum Key Distribution Networks». In: *Entropy* 25.6 (2023), p. 943. DOI: 10.3390/e25060943.

[20] E. Vicario. «Hierarchical SDN control for advanced quantum-secured metropolitan networks». Tesi di laurea mag. Politecnico di Milano, 2025. URL: https://www.politesi.polimi.it/handle/10589/246504.

[21] A. Sebastián-Lombraña et al. *A blueprint for large-scale quantum-network deployments*. arXiv:2409.01069. 2024.

[22] R. B. Méndez et al. «Switching Coordinator: An SDN Application for Flexible QKD Networks». In: *Entropy* 28 (2026). arXiv:2603.13812, p. 219.

[23] S. Laschet et al. «SDN Key Relay for Large-Scale QKD Networks». In: *IEEE OCNC*. arXiv:2608.17539. 2026.

[24] *Interconnecting Regional QKD Networks: Hybrid Key Delivery Across Quantum Domains*. arXiv:2604.20376. 2026.

[25] Martin Bjorklund. *The YANG 1.1 Data Modeling Language*. RFC 7950. IETF, 2016. URL: https://www.rfc-editor.org/rfc/rfc7950.

[26] ETSI ISG QKD. *gs015-ctrl-int-sdn: YANG modules etsi-qkd-sdn-node and etsi-qkd-node-types*. ETSI Forge. 2022. URL: https://forge.etsi.org/rep/qkd/gs015-ctrl-int-sdn.

[27] Ladislav Lhotka. *JSON Encoding of Data Modeled with YANG*. RFC 7951. IETF, 2016. URL: https://www.rfc-editor.org/rfc/rfc7951.

[28] Rob Enns, Martin Bjorklund, Jürgen Schoenwaelder e Andy Bierman. *Network Configuration Protocol (NETCONF)*. RFC 6241. IETF, 2011. URL: https://www.rfc-editor.org/rfc/rfc6241.

[29] Andy Bierman, Martin Bjorklund e Kent Watsen. *RESTCONF Protocol*. RFC 8040. IETF, 2017. URL: https://www.rfc-editor.org/rfc/rfc8040.

[30] Daniele Ceccarelli e Young Lee. *Framework for Abstraction and Control of TE Networks (ACTN)*. RFC 8453. IETF, 2018. URL: https://www.rfc-editor.org/rfc/rfc8453.